MACHINE LEARNING LABORATORY

OBSERVATION RECORD

Candidate-Elimination Algorithm

Five experiments for one question

Question: For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in Python to output a description of the set of all hypotheses consistent with the training examples.

Index of experiments

Part	Data set (CSV file)	Examples	Specific boundary S	General boundary G
1	EnjoySport (enjoysport.csv)	4	<Sunny, Warm, ?, Strong, ?, ?>	<?, Warm, ?, ?, ?, ?> <Sunny, ?, ?, ?, ?, ?>
2	Weather (Play Outdoors) (weather.csv)	5	<Sunny, ?, Normal, ?>	<?, ?, Normal, ?> <Sunny, ?, ?, ?>
3	Medical Diagnosis (Flu) (flu.csv)	5	<High, Yes, ?, ?, ?>	<High, ?, ?, ?, ?>
4	Loan Approval (loan.csv)	6	<?, Good, Salaried, ?>	<?, Good, Salaried, ?>
5	Student Placement (placement.csv)	5	<High, Good, ?, ?, Many>	<?, ?, ?, ?, Many> <?, Good, ?, ?, ?> <High, ?, ?, ?, ?>

Name	JOEL BENN JOHN B
Register No.	1911260003
Department / Year	CSE
Course Code	ITA0614
Academic Year	2026-2030

Each part follows the same eleven-section format: Experiment No. and Date, Experiment Question, Aim, Concept, Dataset Description, Algorithm, Step-by-step Procedure, Formulae, Python Program, Output and Result / Conclusion.

CANDIDATE-ELIMINATION ALGORITHM

Part 1 of 5 | Data set: EnjoySport

1. Experiment No. / Date

Experiment No.: _____ (Part 1 of 5)	Date: ____ / ____ / _____

2. Experiment Question

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in Python to output a description of the set of all hypotheses consistent with the training examples.

Part 1 applies the algorithm to the EnjoySport data set (enjoysport.csv).

3. Aim

To implement the Candidate-Elimination algorithm in Python, read the EnjoySport training examples from the CSV file enjoysport.csv, and output the specific boundary S, the general boundary G and every hypothesis of the version space that is consistent with the training examples.

4. Concept

Concept learning means inferring a Boolean-valued function (the target concept) from labelled training examples. In this experiment a hypothesis is a conjunction of constraints on the attributes, written as a vector $<c_1, c_2, ..., c_n>$. Each constraint c_i is one of three things: a specific value (the attribute must equal that value), ? (any value is acceptable) or 0 (no value is acceptable).

A hypothesis is consistent with the training data if it classifies every training example correctly. The version space is the set of all consistent hypotheses. Instead of listing every member, the Candidate-Elimination algorithm stores only two boundaries: the specific boundary S (the most specific consistent hypotheses) and the general boundary G (the most general consistent hypotheses). Every hypothesis lying between S and G in the general-to-specific ordering also belongs to the version space.

A positive example forces S to become more general (and removes members of G that fail to cover it). A negative example forces G to become more specific (and removes members of S that wrongly cover it). In contrast to Find-S, which tracks only S, Candidate-Elimination also uses the negative examples and therefore describes all consistent hypotheses.

The classic EnjoySport data set from Mitchell (1997), Machine Learning, Chapter 2. Each row describes a day and whether Aldo enjoys his favourite water sport on that day. The training data does not contain every possible value (for example, Sky = Cloudy), so the attribute domains are taken from the values that actually occur in the CSV file.

5. Dataset Description

File: enjoysport.csv. The file has one header row followed by 4 training examples (3 positive and 1 negative). The first 6 columns are the input attributes and the last column is the target: EnjoySport (Yes = Aldo enjoys the sport, No = he does not).

Table 1: Attributes and their values (as observed in the CSV file)

No.	Attribute	Values
1	Sky	Rainy, Sunny
2	AirTemp	Cold, Warm
3	Humidity	High, Normal
4	Wind	Strong
5	Water	Cool, Warm
6	Forecast	Change, Same

Table 2: Training data (enjoysport.csv)

No.	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

6. Algorithm

1. Initialise S to the most specific hypothesis <0, 0, ..., 0> and G to the most general hypothesis <?, ?, ..., ?>.
2. For every training example d = <x, c(x)>, in the order given in the CSV file:

   If d is a positive example:
   (a) Remove from G every hypothesis that does not cover x.
   (b) For each hypothesis s in S that does not cover x: remove s from S, and add all minimal generalisations h of s such that h covers x and some member of G is more general than or equal to h.
   (c) Remove from S every hypothesis that is more general than another hypothesis in S.

   If d is a negative example:
   (a) Remove from S every hypothesis that covers x.
   (b) For each hypothesis g in G that covers x: remove g from G, and add all minimal specialisations h of g such that h does not cover x and some member of S is more specific than or equal to h.
   (c) Remove from G every hypothesis that is less general than another hypothesis in G.
3. After all examples have been processed, output S and G. The version space is every hypothesis h with $g \geq h \geq s$ for some g in G and s in S.

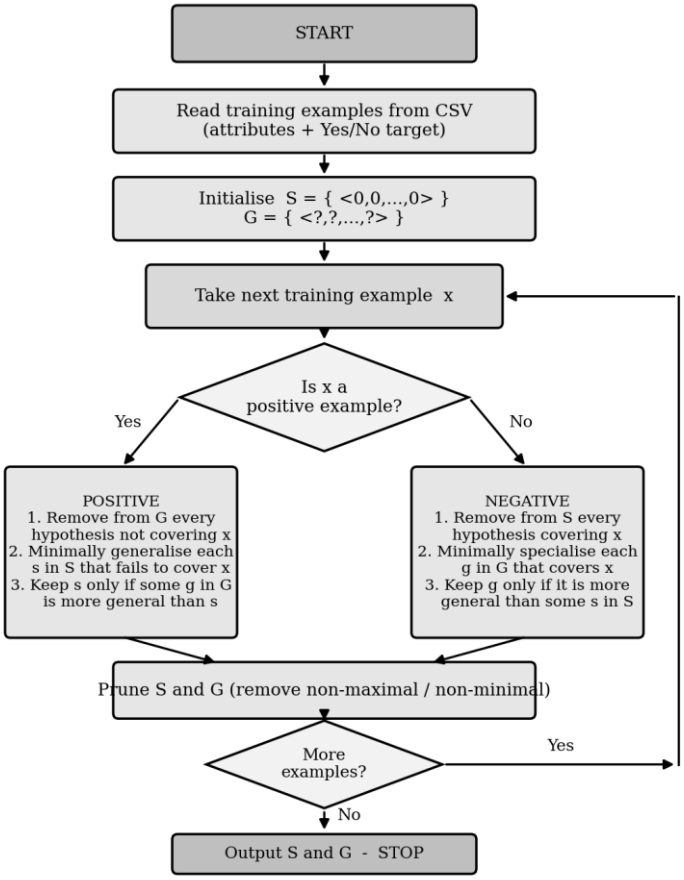


Figure 1: Flowchart of the Candidate-Elimination algorithm

7. Step-by-step Procedure

1. Create a folder for the experiment and save the training data in a CSV file named enjoysport.csv (header row first, target column last).
2. Type the program in a Python editor (IDLE, VS Code or Jupyter) and save it as experiment_1_candidate_elimination_enjoysport.py in the same folder as the CSV file.
3. Function load_data() reads the CSV file and separates the attribute tuples X from the Yes/No labels y.
4. Function candidate_elimination() initialises S = {<0, ..., 0>} and G = {<?, ..., ?>} and finds the domain of each attribute from the data.
5. For every example the function applies the positive-example or negative-example update given in the algorithm, using the helper functions covers(), min_generalization(), min_specializations() and more_general_or_equal().
6. The program prints S and G after every example, which gives the trace shown in Table 3.
7. Function version_space() lists every hypothesis between S and G, and brute_force() enumerates all hypotheses to confirm the same set (verification).
8. Run the program (python experiment_1_candidate_elimination_enjoysport.py), note the output and record the final S, G and version space.

Table 3: Trace of S and G after each training example

No.	Example	Label	Specific boundary S	General boundary G
0	Initial	-	<0, 0, 0, 0, 0, 0>	<?, ?, ?, ?, ?, ?>
1	<Sunny, Warm, Normal, Strong, Warm, Same>	Positive	<Sunny, Warm, Normal, Strong, Warm, Same>	<?, ?, ?, ?, ?, ?>
2	<Sunny, Warm, High, Strong, Warm, Same>	Positive	<Sunny, Warm, ?, Strong, Warm, Same>	<?, ?, ?, ?, ?, ?>
3	<Rainy, Cold, High, Strong, Warm, Change>	Negative	<Sunny, Warm, ?, Strong, Warm, Same>	<?, ?, ?, ?, ?, Same> <?, Warm, ?, ?, ?, ?> <Sunny, ?, ?, ?, ?, ?>
4	<Sunny, Warm, High, Strong, Cool, Change>	Positive	<Sunny, Warm, ?, Strong, ?, ?>	<?, Warm, ?, ?, ?, ?> <Sunny, ?, ?, ?, ?, ?>

8. Formulae

No.	Term	Expression
1	Hypothesis	$h = \langle c_1, c_2, ..., c_n \rangle$, $c_i \in \{ ?, 0, \text{specific value} \}$
2	Hypothesis satisfied by x	$h(x) = 1 \Leftrightarrow$ for every i: $c_i = ?$ or $c_i = x_i$
3	Consistency	Consistent(h, D) $\Leftrightarrow$ $h(x) = c(x)$ for every <x, c(x)> in D
4	More-general-than-or-equal	$h_j \geq_g h_k \Leftrightarrow$ for all x: ($h_k(x) = 1 \rightarrow h_j(x) = 1$)
5	Version space	$VS = \{ h \in H \mid \text{Consistent}(h, D) \}$
6	Specific boundary	$S = \{ s \in H \mid \text{Consistent}(s, D) \wedge \neg\exists\, s' \in H [(s >_g s') \wedge \text{Consistent}(s', D)] \}$
7	General boundary	$G = \{ g \in H \mid \text{Consistent}(g, D) \wedge \neg\exists\, g' \in H [(g' >_g g) \wedge \text{Consistent}(g', D)] \}$
8	Version-space representation	$VS = \{ h \in H \mid \exists\, s \in S, \exists\, g \in G : g \geq_g h \geq_g s \}$
9	Minimal generalisation (positive x)	$c_i' = x_i$ if $c_i = 0$; $c_i' = c_i$ if $c_i = x_i$ or $c_i = ?$; $c_i' = ?$ otherwise
10	Minimal specialisation (negative x)	for each i with $g_i = ?$, replace g_i by every value $v \neq x_i$ in the domain of attribute i
11	Syntactically distinct hypotheses	$\prod (\lvert V_i \rvert + 2) = (2+2)\times(2+2)\times(2+2)\times(1+2)\times(2+2)\times(2+2) = 3072$
12	Semantically distinct hypotheses	$1 + \prod (\lvert V_i \rvert + 1) = 1 + (2+1)\times(2+1)\times(2+1)\times(1+1)\times(2+1)\times(2+1) = 487$

9. Python Program

File name: experiment_1_candidate_elimination_enjoysport.py Input file: enjoysport.csv

```
"""
Experiment 1 : Candidate-Elimination Algorithm
Dataset          : EnjoySport (Mitchell)  (enjoysport.csv)

Reads training examples from a CSV file (last column = target, "Yes"/"No")
and prints the version space: the specific boundary S and the general
boundary G, i.e. all hypotheses consistent with the training examples.
"""
```

```
import csv
from itertools import product

CSV_FILE = "enjoysport.csv"



# ---------------------------------------------------------------- helpers
def load_data(path):
    """Return attribute names, list of attribute tuples, list of booleans."""
    with open(path, newline="") as f:
        rows = list(csv.reader(f))
    header, data = rows[0], rows[1:]
    X = [tuple(v.strip() for v in r[:-1]) for r in data]
    y = [r[-1].strip().lower() == "yes" for r in data]
    return header[:-1], X, y



def show(h):
    return "<" + ", ".join(h) + ">"



def show_set(hs):
    return "{ " + ", ".join(show(h) for h in sorted(hs)) + " }" if hs else "{ }"



def covers(h, x):
    """h(x) = 1 when every constraint of h is '?' or equals the value in x."""
    return all(c == "?" or c == v for c, v in zip(h, x))



def more_general_or_equal(h1, h2):
    """True if h1 >=g h2 (h1 covers everything that h2 covers)."""
    if "0" in h2:                       # h2 covers nothing
        return True
    return all(a == "?" or a == b for a, b in zip(h1, h2))



def strictly_more_general(h1, h2):
    return more_general_or_equal(h1, h2) and not more_general_or_equal(h2, h1)



def min_generalization(s, x):
    """Minimal generalisation of s that covers the positive example x."""
    new = []
    for c, v in zip(s, x):
        if c == "0":
            new.append(v)
        elif c == v or c == "?":
            new.append(c)
        else:
            new.append("?")
    return tuple(new)



def min_specializations(g, x, domains):
    """Minimal specialisations of g that exclude the negative example x."""
    out = []
    for i, c in enumerate(g):
        if c == "?":
            for v in domains[i]:
                if v != x[i]:
                    out.append(g[:i] + (v,) + g[i + 1:])
    return out



# ---------------------------------------------------------- main algorithm
def candidate_elimination(X, y):
```

```
    n = len(X[0])
    domains = [sorted({x[i] for x in X}) for i in range(n)]
    S = {("0",) * n}                  # most specific boundary
    G = {("?",) * n}                  # most general boundary
    trace = []

    for t, (x, positive) in enumerate(zip(X, y), start=1):
        if positive:
            G = {g for g in G if covers(g, x)}
            for s in list(S):
                if not covers(s, x):
                    S.remove(s)
                    h = min_generalization(s, x)
                    if any(more_general_or_equal(g, h) for g in G):
                        S.add(h)
            S = {s for s in S
                 if not any(strictly_more_general(s, o) for o in S)}
        else:
            S = {s for s in S if not covers(s, x)}
            for g in list(G):
                if covers(g, x):
                    G.remove(g)
                    for h in min_specializations(g, x, domains):
                        if any(more_general_or_equal(h, s) for s in S):
                            G.add(h)
            G = {g for g in G
                 if not any(strictly_more_general(o, g) for o in G)}
        trace.append((t, x, positive, set(S), set(G)))
    return S, G, trace, domains



def version_space(S, G, domains):
    """All hypotheses lying between S and G (brute-force enumeration)."""
    space = []
    for h in product(*[d + ["?"] for d in domains]):
        if any(more_general_or_equal(g, h) for g in G) and \
           any(more general or equal(h, s) for s in S):
            space.append(h)
    return space



def brute_force(X, y, domains):
    """All hypotheses consistent with every example (used to verify)."""
    return [h for h in product(*[d + ["?"] for d in domains])
            if all(covers(h, x) == lab for x, lab in zip(X, y))]



# ---------------------------------------------------------------- driver
if __name__ == "__main__":
    attrs, X, y = load_data(CSV_FILE)
    print("Attributes :", attrs)
    print("Examples   :", len(X), "(", sum(y), "positive,",
          len(y) - sum(y), "negative )\n")

    print("Initial  S0 =", show_set({("0",) * len(attrs)}))
    print("Initial  G0 =", show_set({("?",) * len(attrs)}), "\n")

    S, G, trace, domains = candidate_elimination(X, y)

    for t, x, positive, s_t, g_t in trace:
        label = "Positive" if positive else "Negative"
        print(f"Example {t}: {show(x)} -> {label}")
        print(f"   S{t} =", show_set(s_t))
        print(f"   G{t} =", show_set(g_t))

    print("\nFinal specific boundary S :", show_set(S))
    print("Final general  boundary G :", show_set(G))

    vs = version_space(S, G, domains)
    print(f"\nVersion space ({len(vs)} hypotheses consistent with the data):")
    for h in sorted(vs, key=lambda h: (h.count("?"), h)):
        print("   ", show(h))
```

```
    ok = sorted(vs) == sorted(brute_force(X, y, domains))
    print("\nVerification against brute-force search :", "PASSED" if ok else "FAILED")
```

10. Output

Console output of experiment_1_candidate_elimination_enjoysport.py:

```
Attributes : ['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast']
Examples   : 4 ( 3 positive, 1 negative )


Initial  S0 = { <0, 0, 0, 0, 0, 0> }
Initial  G0 = { <?, ?, ?, ?, ?, ?> }


Example 1: <Sunny, Warm, Normal, Strong, Warm, Same> -> Positive
   S1 = { <Sunny, Warm, Normal, Strong, Warm, Same> }
   G1 = { <?, ?, ?, ?, ?, ?> }
Example 2: <Sunny, Warm, High, Strong, Warm, Same> -> Positive
   S2 = { <Sunny, Warm, ?, Strong, Warm, Same> }
   G2 = { <?, ?, ?, ?, ?, ?> }
Example 3: <Rainy, Cold, High, Strong, Warm, Change> -> Negative
   S3 = { <Sunny, Warm, ?, Strong, Warm, Same> }
   G3 = { <?, ?, ?, ?, ?, Same>, <?, Warm, ?, ?, ?, ?>, <Sunny, ?, ?, ?, ?, ?> }
Example 4: <Sunny, Warm, High, Strong, Cool, Change> -> Positive
   S4 = { <Sunny, Warm, ?, Strong, ?, ?> }
   G4 = { <?, Warm, ?, ?, ?, ?>, <Sunny, ?, ?, ?, ?, ?> }


Final specific boundary S : { <Sunny, Warm, ?, Strong, ?, ?> }
Final general  boundary G : { <?, Warm, ?, ?, ?, ?>, <Sunny, ?, ?, ?, ?, ?> }


Version space (6 hypotheses consistent with the data):
    <Sunny, Warm, ?, Strong, ?, ?>
    <?, Warm, ?, Strong, ?, ?>
    <Sunny, ?, ?, Strong, ?, ?>
    <Sunny, Warm, ?, ?, ?, ?>
    <?, Warm, ?, ?, ?, ?>
    <Sunny, ?, ?, ?, ?, ?>


Verification against brute-force search : PASSED
```

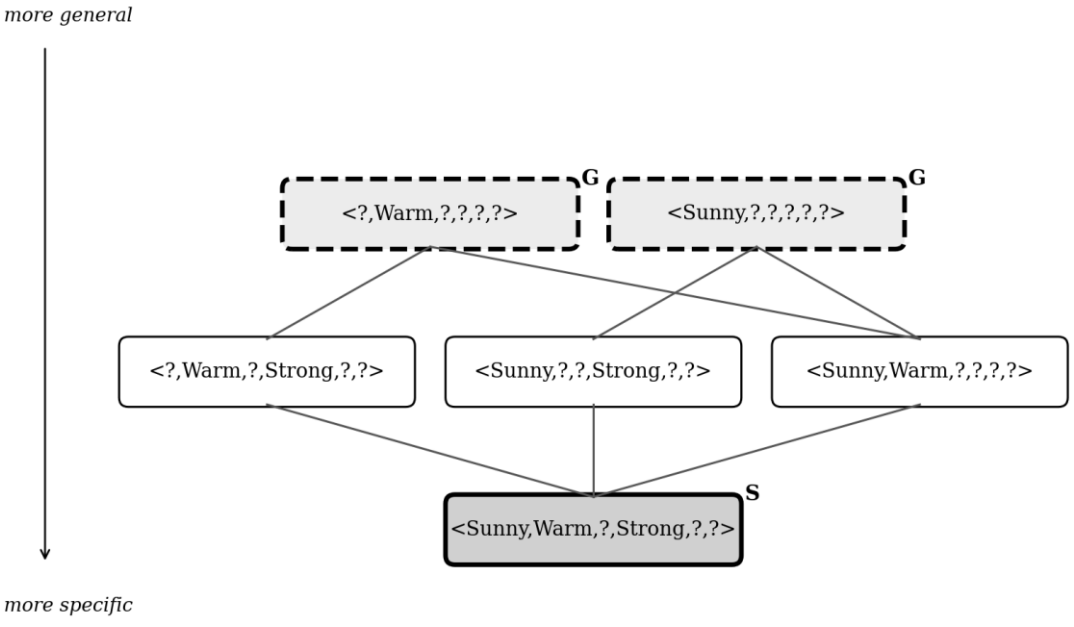


Figure 2: Version space for the EnjoySport data (S = specific boundary, G = general boundary)

11. Result / Conclusion

The Candidate-Elimination algorithm was implemented in Python and run on the EnjoySport training data (4 examples, 6 attributes). The version space contains 6 consistent hypotheses.

Final specific boundary S: { <Sunny, Warm, ?, Strong, ?, ?> }

Final general boundary G: { <?, Warm, ?, ?, ?, ?>, <Sunny, ?, ?, ?, ?, ?> }

The result agrees with the textbook solution: S4 = {<Sunny, Warm, ?, Strong, ?, ?>} and G4 = {<Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>}. The negative example (Example 3) split G into three members, and the final positive example (Example 4) generalised S and removed <?, ?, ?, ?, ?, Same> from G because it did not cover that example.

The version space found by the algorithm was identical to the set of hypotheses found by exhaustive search, so the implementation is verified. Hence the Candidate-Elimination algorithm outputs a description of the set of all hypotheses consistent with the training examples.

Evaluation rubric (to be filled in by the staff member)

Criteria	Max. marks	Marks awarded
Aim, concept and dataset description	10	
Algorithm, flowchart and procedure	10	
Python program (correctness and style)	15	
Output and verification	10	
Result / conclusion	5	
Total	50	

Student signature: _____ Staff signature: _____

CANDIDATE-ELIMINATION ALGORITHM

Part 2 of 5 | Data set: Weather (Play Outdoors)

1. Experiment No. / Date

Experiment No.: _____ (Part 2 of 5)	Date: ____ / ____ / _____

2. Experiment Question

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in Python to output a description of the set of all hypotheses consistent with the training examples.

Part 2 applies the algorithm to the Weather (Play Outdoors) data set (weather.csv).

3. Aim

To implement the Candidate-Elimination algorithm in Python, read the Weather (Play Outdoors) training examples from the CSV file weather.csv, and output the specific boundary S, the general boundary G and every hypothesis of the version space that is consistent with the training examples.

4. Concept

Concept learning means inferring a Boolean-valued function (the target concept) from labelled training examples. In this experiment a hypothesis is a conjunction of constraints on the attributes, written as a vector $<c_1, c_2, ..., c_n>$. Each constraint c_i is one of three things: a specific value (the attribute must equal that value), ? (any value is acceptable) or 0 (no value is acceptable).

A hypothesis is consistent with the training data if it classifies every training example correctly. The version space is the set of all consistent hypotheses. Instead of listing every member, the Candidate-Elimination algorithm stores only two boundaries: the specific boundary S (the most specific consistent hypotheses) and the general boundary G (the most general consistent hypotheses). Every hypothesis lying between S and G in the general-to-specific ordering also belongs to the version space.

A positive example forces S to become more general (and removes members of G that fail to cover it). A negative example forces G to become more specific (and removes members of S that wrongly cover it). In contrast to Find-S, which tracks only S, Candidate-Elimination also uses the negative examples and therefore describes all consistent hypotheses.

A small weather data set in which each row describes the weather of a day and whether outdoor play was possible. Three attributes have three values and one has two, so the hypothesis space is still small enough to verify by hand.

5. Dataset Description

File: weather.csv. The file has one header row followed by 5 training examples (3 positive and 2 negative). The first 4 columns are the input attributes and the last column is the target: PlayOutdoors (Yes / No).

Table 1: Attributes and their values (as observed in the CSV file)

No.	Attribute	Values
1	Outlook	Overcast, Rain, Sunny
2	Temperature	Hot, Mild
3	Humidity	High, Normal
4	Wind	Strong, Weak

Table 2: Training data (weather.csv)

No.	Outlook	Temperature	Humidity	Wind	PlayOutdoors
1	Sunny	Mild	Normal	Weak	Yes
2	Sunny	Hot	Normal	Strong	Yes
3	Rain	Mild	High	Weak	No
4	Overcast	Hot	High	Weak	No
5	Sunny	Mild	Normal	Strong	Yes

6. Algorithm

1. Initialise S to the most specific hypothesis <0, 0, ..., 0> and G to the most general hypothesis <?, ?, ..., ?>.
2. For every training example d = <x, c(x)>, in the order given in the CSV file:

If d is a positive example:

(d) Remove from G every hypothesis that does not cover x.

(e) For each hypothesis s in S that does not cover x: remove s from S, and add all minimal generalisations h of s such that h covers x and some member of G is more general than or equal to h.

(f) Remove from S every hypothesis that is more general than another hypothesis in S.

If d is a negative example:

(d) Remove from S every hypothesis that covers x.

(e) For each hypothesis g in G that covers x: remove g from G, and add all minimal specialisations h of g such that h does not cover x and some member of S is more specific than or equal to h.

(f) Remove from G every hypothesis that is less general than another hypothesis in G.

3. After all examples have been processed, output S and G. The version space is every hypothesis h with $g \geq h \geq s$ for some g in G and s in S.

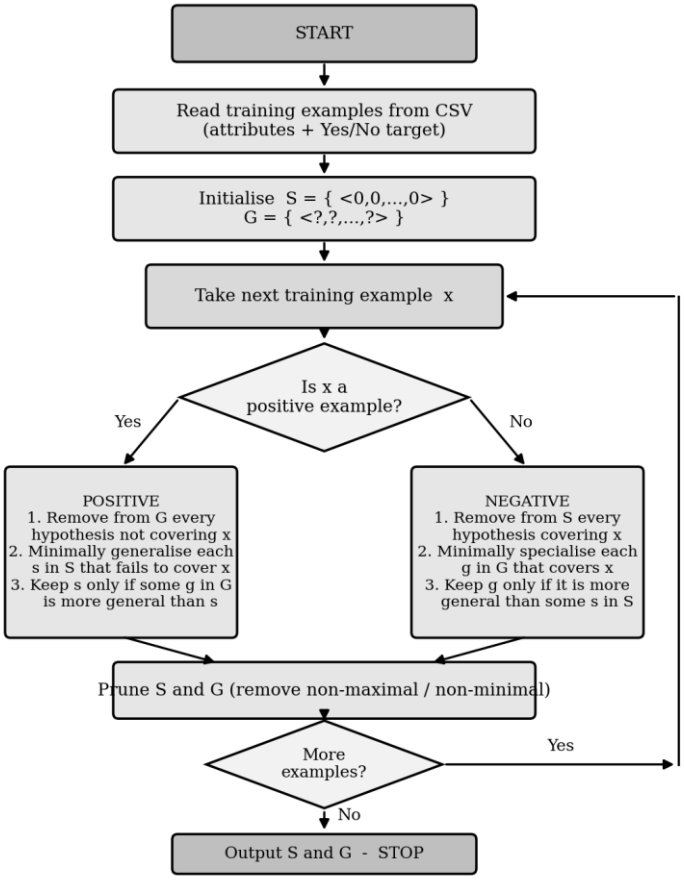


Figure 1: Flowchart of the Candidate-Elimination algorithm

7. Step-by-step Procedure

1. Create a folder for the experiment and save the training data in a CSV file named weather.csv (header row first, target column last).
2. Type the program in a Python editor (IDLE, VS Code or Jupyter) and save it as experiment_2_candidate_elimination_weather.py in the same folder as the CSV file.
3. Function load_data() reads the CSV file and separates the attribute tuples X from the Yes/No labels y.
4. Function candidate_elimination() initialises S = {<0, ..., 0>} and G = {<?, ..., ?>} and finds the domain of each attribute from the data.
5. For every example the function applies the positive-example or negative-example update given in the algorithm, using the helper functions covers(), min_generalization(), min_specializations() and more_general_or_equal().
6. The program prints S and G after every example, which gives the trace shown in Table 3.
7. Function version_space() lists every hypothesis between S and G, and brute_force() enumerates all hypotheses to confirm the same set (verification).
8. Run the program (python experiment_2_candidate_elimination_weather.py), note the output and record the final S, G and version space.

Table 3: Trace of S and G after each training example

No.	Example	Label	Specific boundary S	General boundary G
0	Initial	-	<0, 0, 0, 0>	<?, ?, ?, ?>
1	<Sunny, Mild, Normal, Weak>	Positive	<Sunny, Mild, Normal, Weak>	<?, ?, ?, ?>
2	<Sunny, Hot, Normal, Strong>	Positive	<Sunny, ?, Normal, ?>	<?, ?, ?, ?>
3	<Rain, Mild, High, Weak>	Negative	<Sunny, ?, Normal, ?>	<?, ?, Normal, ?> <Sunny, ?, ?, ?>
4	<Overcast, Hot, High, Weak>	Negative	<Sunny, ?, Normal, ?>	<?, ?, Normal, ?> <Sunny, ?, ?, ?>
5	<Sunny, Mild, Normal, Strong>	Positive	<Sunny, ?, Normal, ?>	<?, ?, Normal, ?> <Sunny, ?, ?, ?>

8. Formulae

No.	Term	Expression
1	Hypothesis	$h = \langle c_1, c_2, ..., c_n \rangle$, $c_i \in \{ ?, 0, \text{specific value} \}$
2	Hypothesis satisfied by x	$h(x) = 1 \iff$ for every i: $c_i = ?$ or $c_i = x_i$
3	Consistency	Consistent(h, D) $\iff h(x) = c(x)$ for every <x, c(x)> in D
4	More-general-than-or-equal	$h_j \geq_g h_k \iff$ for all x: ($h_k(x) = 1 \rightarrow h_j(x) = 1$)
5	Version space	$VS = \{ h \in H \mid \text{Consistent}(h, D) \}$
6	Specific boundary	$S = \{ s \in H \mid \text{Consistent}(s, D) \wedge \neg\exists\, s' \in H [(s >_g s') \wedge \text{Consistent}(s', D)] \}$
7	General boundary	$G = \{ g \in H \mid \text{Consistent}(g, D) \wedge \neg\exists\, g' \in H [(g' >_g g) \wedge \text{Consistent}(g', D)] \}$
8	Version-space representation	$VS = \{ h \in H \mid \exists\, s \in S, \exists\, g \in G : g \geq_g h \geq_g s \}$
9	Minimal generalisation (positive x)	$c_i' = x_i$ if $c_i = 0$; $c_i' = c_i$ if $c_i = x_i$ or $c_i = ?$; $c_i' = ?$ otherwise
10	Minimal specialisation (negative x)	for each i with $g_i = ?$, replace g_i by every value $v \neq x_i$ in the domain of attribute i
11	Syntactically distinct hypotheses	$\prod (\lvert V_i \rvert + 2) = (3+2)\times(2+2)\times(2+2)\times(2+2) = 320$
12	Semantically distinct hypotheses	$1 + \prod (\lvert V_i \rvert + 1) = 1 + (3+1)\times(2+1)\times(2+1)\times(2+1) = 109$

9. Python Program

File name: experiment_2_candidate_elimination_weather.py Input file: weather.csv

```
"""
Experiment 2 : Candidate-Elimination Algorithm
Dataset          : Weather - Play Outdoors  (weather.csv)

Reads training examples from a CSV file (last column = target, "Yes"/"No")
and prints the version space: the specific boundary S and the general
```

```
boundary G, i.e. all hypotheses consistent with the training examples.
"""


import csv
from itertools import product


CSV_FILE = "weather.csv"



# ---------------------------------------------------------------- helpers
def load_data(path):
    """Return attribute names, list of attribute tuples, list of booleans."""
    with open(path, newline="") as f:
        rows = list(csv.reader(f))
    header, data = rows[0], rows[1:]
    X = [tuple(v.strip() for v in r[:-1]) for r in data]
    y = [r[-1].strip().lower() == "yes" for r in data]
    return header[:-1], X, y



def show(h):
    return "<" + ", ".join(h) + ">"



def show_set(hs):
    return "{ " + ", ".join(show(h) for h in sorted(hs)) + " }" if hs else "{ }"



def covers(h, x):
    """h(x) = 1 when every constraint of h is '?' or equals the value in x."""
    return all(c == "?" or c == v for c, v in zip(h, x))



def more_general_or_equal(h1, h2):
    """True if h1 >=g h2 (h1 covers everything that h2 covers)."""
    if "0" in h2:                       # h2 covers nothing
        return True
    return all(a == "?" or a == b for a, b in zip(h1, h2))



def strictly_more_general(h1, h2):
    return more_general_or_equal(h1, h2) and not more_general_or_equal(h2, h1)



def min_generalization(s, x):
    """Minimal generalisation of s that covers the positive example x."""
    new = []
    for c, v in zip(s, x):
        if c == "0":
            new.append(v)
        elif c == v or c == "?":
            new.append(c)
        else:
            new.append("?")
    return tuple(new)



def min_specializations(g, x, domains):
    """Minimal specialisations of g that exclude the negative example x."""
    out = []
    for i, c in enumerate(g):
        if c == "?":
            for v in domains[i]:
                if v != x[i]:
                    out.append(g[:i] + (v,) + g[i + 1:])
    return out
```

```
# ------------------------------------------------------ main algorithm
def candidate_elimination(X, y):
    n = len(X[0])
    domains = [sorted({x[i] for x in X}) for i in range(n)]
    S = {("0",) * n}                    # most specific boundary
    G = {("?",) * n}                    # most general boundary
    trace = []

    for t, (x, positive) in enumerate(zip(X, y), start=1):
        if positive:
            G = {g for g in G if covers(g, x)}
            for s in list(S):
                if not covers(s, x):
                    S.remove(s)
                    h = min_generalization(s, x)
                    if any(more_general_or_equal(g, h) for g in G):
                        S.add(h)
            S = {s for s in S
                 if not any(strictly_more_general(s, o) for o in S)}
        else:
            S = {s for s in S if not covers(s, x)}
            for g in list(G):
                if covers(g, x):
                    G.remove(g)
                    for h in min_specializations(g, x, domains):
                        if any(more_general_or_equal(h, s) for s in S):
                            G.add(h)
            G = {g for g in G
                 if not any(strictly_more_general(o, g) for o in G)}
        trace.append((t, x, positive, set(S), set(G)))
    return S, G, trace, domains



def version_space(S, G, domains):
    """All hypotheses lying between S and G (brute-force enumeration)."""
    space = []
    for h in product(*[d + ["?"] for d in domains]):
        if any(more_general_or_equal(g, h) for g in G) and \
           any(more_general_or_equal(h, s) for s in S):
            space.append(h)
    return space



def brute_force(X, y, domains):
    """All hypotheses consistent with every example (used to verify)."""
    return [h for h in product(*[d + ["?"] for d in domains])
            if all(covers(h, x) == lab for x, lab in zip(X, y))]



# ------------------------------------------------------------ driver
if   name   == "  main  ":
    attrs, X, y = load_data(CSV_FILE)
    print("Attributes :", attrs)
    print("Examples   :", len(X), "(", sum(y), "positive,",
          len(y) - sum(y), "negative )\n")

    print("Initial  S0 =", show_set({("0",) * len(attrs)}))
    print("Initial  G0 =", show_set({("?",) * len(attrs)}), "\n")

    S, G, trace, domains = candidate_elimination(X, y)

    for t, x, positive, s_t, g_t in trace:
        label = "Positive" if positive else "Negative"
        print(f"Example {t}: {show(x)} -> {label}")
        print(f"   S{t} =", show_set(s_t))
        print(f"   G{t} =", show_set(g_t))

    print("\nFinal specific boundary S :", show_set(S))
    print("Final general  boundary G :", show_set(G))

    vs = version_space(S, G, domains)
    print(f"\nVersion space ({len(vs)} hypotheses consistent with the data):")
```

```
    for h in sorted(vs, key=lambda h: (h.count("?"), h)):
        print("   ", show(h))

    ok = sorted(vs) == sorted(brute_force(X, y, domains))
    print("\nVerification against brute-force search :", "PASSED" if ok else "FAILED")
```

10. Output

Console output of experiment_2_candidate_elimination_weather.py:

```
Attributes : ['Outlook', 'Temperature', 'Humidity', 'Wind']
Examples   : 5 ( 3 positive, 2 negative )


Initial  S0 = { <0, 0, 0, 0> }
Initial  G0 = { <?, ?, ?, ?> }


Example 1: <Sunny, Mild, Normal, Weak> -> Positive
   S1 = { <Sunny, Mild, Normal, Weak> }
   G1 = { <?, ?, ?, ?> }
Example 2: <Sunny, Hot, Normal, Strong> -> Positive
   S2 = { <Sunny, ?, Normal, ?> }
   G2 = { <?, ?, ?, ?> }
Example 3: <Rain, Mild, High, Weak> -> Negative
   S3 = { <Sunny, ?, Normal, ?> }
   G3 = { <?, ?, Normal, ?>, <Sunny, ?, ?, ?> }
Example 4: <Overcast, Hot, High, Weak> -> Negative
   S4 = { <Sunny, ?, Normal, ?> }
   G4 = { <?, ?, Normal, ?>, <Sunny, ?, ?, ?> }
Example 5: <Sunny, Mild, Normal, Strong> -> Positive
   S5 = { <Sunny, ?, Normal, ?> }
   G5 = { <?, ?, Normal, ?>, <Sunny, ?, ?, ?> }


Final specific boundary S : { <Sunny, ?, Normal, ?> }
Final general  boundary G : { <?, ?, Normal, ?>, <Sunny, ?, ?, ?> }


Version space (3 hypotheses consistent with the data):
    <Sunny, ?, Normal, ?>
    <?, ?, Normal, ?>
    <Sunny, ?, ?, ?>


Verification against brute-force search : PASSED
```

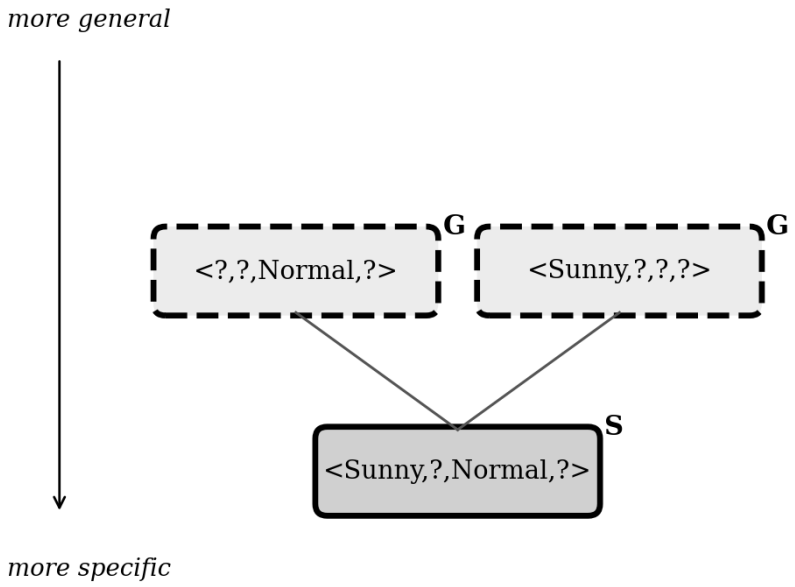


Figure 2: Version space for the Weather (Play Outdoors) data (S = specific boundary, G = general boundary)

11. Result / Conclusion

The Candidate-Elimination algorithm was implemented in Python and run on the Weather (Play Outdoors) training data (5 examples, 4 attributes). The version space contains 3 consistent hypotheses.

Final specific boundary S: { <Sunny, ?, Normal, ?> }

Final general boundary G: { <?, ?, Normal, ?>, <Sunny, ?, ?, ?> }

Examples 4 (negative) and 5 (positive) caused no change in S or G: the negative example was already excluded by every member of G and the positive example was already covered by S. This shows that the algorithm only changes the version space when an example is informative.

The version space found by the algorithm was identical to the set of hypotheses found by exhaustive search, so the implementation is verified. Hence the Candidate-Elimination algorithm outputs a description of the set of all hypotheses consistent with the training examples.

Evaluation rubric (to be filled in by the staff member)

Criteria	Max. marks	Marks awarded
Aim, concept and dataset description	10	
Algorithm, flowchart and procedure	10	
Python program (correctness and style)	15	
Output and verification	10	
Result / conclusion	5	
Total	50	

Student signature: _____ Staff signature: _____

CANDIDATE-ELIMINATION ALGORITHM

Part 3 of 5 | Data set: Medical Diagnosis (Flu)

1. Experiment No. / Date

Experiment No.: _____ (Part 3 of 5)	Date: ____ / ____ / _____

2. Experiment Question

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in Python to output a description of the set of all hypotheses consistent with the training examples.

Part 3 applies the algorithm to the Medical Diagnosis (Flu) data set (flu.csv).

3. Aim

To implement the Candidate-Elimination algorithm in Python, read the Medical Diagnosis (Flu) training examples from the CSV file flu.csv, and output the specific boundary S, the general boundary G and every hypothesis of the version space that is consistent with the training examples.

4. Concept

Concept learning means inferring a Boolean-valued function (the target concept) from labelled training examples. In this experiment a hypothesis is a conjunction of constraints on the attributes, written as a vector $<c_1, c_2, ..., c_n>$. Each constraint c_i is one of three things: a specific value (the attribute must equal that value), ? (any value is acceptable) or 0 (no value is acceptable).

A hypothesis is consistent with the training data if it classifies every training example correctly. The version space is the set of all consistent hypotheses. Instead of listing every member, the Candidate-Elimination algorithm stores only two boundaries: the specific boundary S (the most specific consistent hypotheses) and the general boundary G (the most general consistent hypotheses). Every hypothesis lying between S and G in the general-to-specific ordering also belongs to the version space.

A positive example forces S to become more general (and removes members of G that fail to cover it). A negative example forces G to become more specific (and removes members of S that wrongly cover it). In contrast to Find-S, which tracks only S, Candidate-Elimination also uses the negative examples and therefore describes all consistent hypotheses.

A simple medical diagnosis data set. Each row records the symptoms of a patient and whether the patient was diagnosed with flu. The data is illustrative and is used only to demonstrate the algorithm. This part uses five attributes with mixed domain sizes (two values for four attributes and three for BodyAche).

5. Dataset Description

File: flu.csv. The file has one header row followed by 5 training examples (3 positive and 2 negative). The first 5 columns are the input attributes and the last column is the target: Flu (Yes = flu diagnosed, No = not flu).

Table 1: Attributes and their values (as observed in the CSV file)

No.	Attribute	Values
1	Fever	High, Normal
2	Cough	No, Yes
3	Fatigue	No, Yes
4	BodyAche	Mild, None, Severe
5	Rash	No, Yes

Table 2: Training data (flu.csv)

No.	Fever	Cough	Fatigue	BodyAche	Rash	Flu
1	High	Yes	Yes	Severe	No	Yes
2	High	Yes	No	Severe	No	Yes
3	Normal	No	Yes	Mild	No	No
4	High	Yes	Yes	Mild	Yes	Yes
5	Normal	Yes	No	None	No	No

6. Algorithm

1. Initialise S to the most specific hypothesis <0, 0, ..., 0> and G to the most general hypothesis <?, ?, ..., ?>.
2. For every training example d = <x, c(x)>, in the order given in the CSV file:

   If d is a positive example:
   - (g) Remove from G every hypothesis that does not cover x.
   - (h) For each hypothesis s in S that does not cover x: remove s from S, and add all minimal generalisations h of s such that h covers x and some member of G is more general than or equal to h.
   - (i) Remove from S every hypothesis that is more general than another hypothesis in S.

   If d is a negative example:
   - (g) Remove from S every hypothesis that covers x.
   - (h) For each hypothesis g in G that covers x: remove g from G, and add all minimal specialisations h of g such that h does not cover x and some member of S is more specific than or equal to h.
   - (i) Remove from G every hypothesis that is less general than another hypothesis in G.
3. After all examples have been processed, output S and G. The version space is every hypothesis h with $g \geq h \geq s$ for some g in G and s in S.

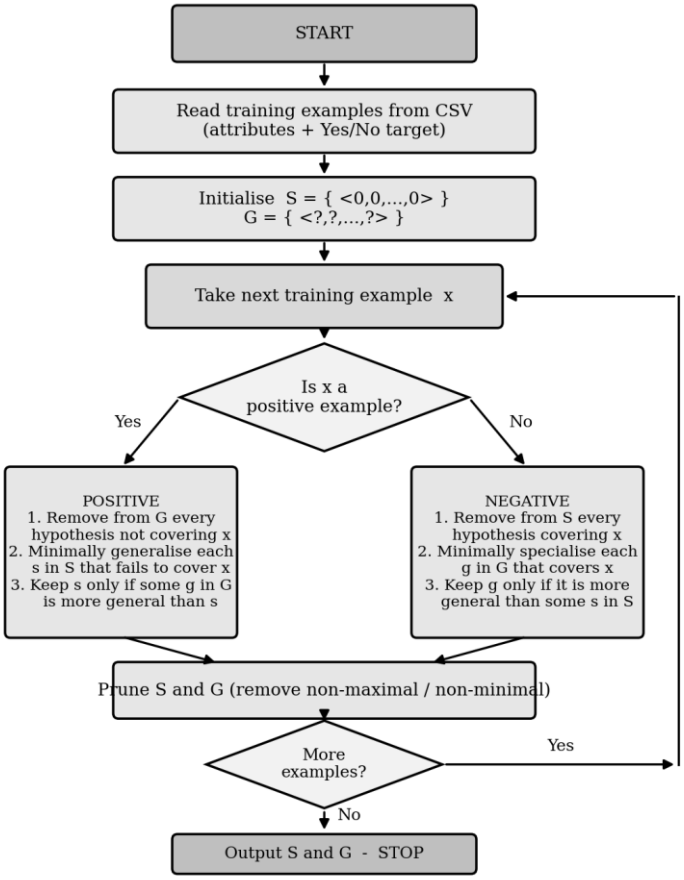


Figure 1: Flowchart of the Candidate-Elimination algorithm

7. Step-by-step Procedure

1. Create a folder for the experiment and save the training data in a CSV file named flu.csv (header row first, target column last).
2. Type the program in a Python editor (IDLE, VS Code or Jupyter) and save it as experiment_3_candidate_elimination_flu.py in the same folder as the CSV file.
3. Function load_data() reads the CSV file and separates the attribute tuples X from the Yes/No labels y.
4. Function candidate_elimination() initialises S = {<0, ..., 0>} and G = {<?, ..., ?>} and finds the domain of each attribute from the data.
5. For every example the function applies the positive-example or negative-example update given in the algorithm, using the helper functions covers(), min_generalization(), min_specializations() and more_general_or_equal().
6. The program prints S and G after every example, which gives the trace shown in Table 3.
7. Function version_space() lists every hypothesis between S and G, and brute_force() enumerates all hypotheses to confirm the same set (verification).
8. Run the program (python experiment_3_candidate_elimination_flu.py), note the output and record the final S, G and version space.

Table 3: Trace of S and G after each training example

No.	Example	Label	Specific boundary S	General boundary G
0	Initial	-	<0, 0, 0, 0, 0>	<?, ?, ?, ?, ?>
1	<High, Yes, Yes, Severe, No>	Positive	<High, Yes, Yes, Severe, No>	<?, ?, ?, ?, ?>
2	<High, Yes, No, Severe, No>	Positive	<High, Yes, ?, Severe, No>	<?, ?, ?, ?, ?>
3	<Normal, No, Yes, Mild, No>	Negative	<High, Yes, ?, Severe, No>	<?, ?, ?, Severe, ?> <?, Yes, ?, ?, ?> <High, ?, ?, ?, ?>
4	<High, Yes, Yes, Mild, Yes>	Positive	<High, Yes, ?, ?, ?>	<?, Yes, ?, ?, ?> <High, ?, ?, ?, ?>
5	<Normal, Yes, No, None, No>	Negative	<High, Yes, ?, ?, ?>	<High, ?, ?, ?, ?>

8. Formulae

No.	Term	Expression
1	Hypothesis	$h = \langle c_1, c_2, ..., c_n \rangle$, $c_i \in \{ ?, 0, \text{specific value} \}$
2	Hypothesis satisfied by x	$h(x) = 1 \iff$ for every i: $c_i = ?$ or $c_i = x_i$
3	Consistency	Consistent(h, D) $\iff h(x) = c(x)$ for every <x, c(x)> in D
4	More-general-than-or-equal	$h_j \geq_g h_k \iff$ for all x: ($h_k(x) = 1 \rightarrow h_j(x) = 1$)
5	Version space	$VS = \{ h \in H \mid \text{Consistent}(h, D) \}$
6	Specific boundary	$S = \{ s \in H \mid \text{Consistent}(s, D) \wedge \neg\exists\, s' \in H [(s >_g s') \wedge \text{Consistent}(s', D)] \}$
7	General boundary	$G = \{ g \in H \mid \text{Consistent}(g, D) \wedge \neg\exists\, g' \in H [(g' >_g g) \wedge \text{Consistent}(g', D)] \}$
8	Version-space representation	$VS = \{ h \in H \mid \exists\, s \in S, \exists\, g \in G : g \geq_g h \geq_g s \}$
9	Minimal generalisation (positive x)	$c_i' = x_i$ if $c_i = 0$; $c_i' = c_i$ if $c_i = x_i$ or $c_i = ?$; $c_i' = ?$ otherwise
10	Minimal specialisation (negative x)	for each i with $g_i = ?$, replace g_i by every value $v \neq x_i$ in the domain of attribute i
11	Syntactically distinct hypotheses	$\prod (\lvert V_i \rvert + 2) = (2+2)\times(2+2)\times(2+2)\times(3+2)\times(2+2) = 1280$
12	Semantically distinct hypotheses	$1 + \prod (\lvert V_i \rvert + 1) = 1 + (2+1)\times(2+1)\times(2+1)\times(3+1)\times(2+1) = 325$

9. Python Program

File name: experiment_3_candidate_elimination_flu.py Input file: flu.csv

```
"""
Experiment 3 : Candidate-Elimination Algorithm
Dataset          : Medical Diagnosis - Flu  (flu.csv)

Reads training examples from a CSV file (last column = target, "Yes"/"No")
```

```
and prints the version space: the specific boundary S and the general
boundary G, i.e. all hypotheses consistent with the training examples.
"""


import csv
from itertools import product


CSV_FILE = "flu.csv"



# ---------------------------------------------------------------- helpers
def load_data(path):
    """Return attribute names, list of attribute tuples, list of booleans."""
    with open(path, newline="") as f:
        rows = list(csv.reader(f))
    header, data = rows[0], rows[1:]
    X = [tuple(v.strip() for v in r[:-1]) for r in data]
    y = [r[-1].strip().lower() == "yes" for r in data]
    return header[:-1], X, y



def show(h):
    return "<" + ", ".join(h) + ">"



def show_set(hs):
    return "{ " + ", ".join(show(h) for h in sorted(hs)) + " }" if hs else "{ }"



def covers(h, x):
    """h(x) = 1 when every constraint of h is '?' or equals the value in x."""
    return all(c == "?" or c == v for c, v in zip(h, x))



def more_general_or_equal(h1, h2):
    """True if h1 >=g h2 (h1 covers everything that h2 covers)."""
    if "0" in h2:                       # h2 covers nothing
        return True
    return all(a == "?" or a == b for a, b in zip(h1, h2))



def strictly_more_general(h1, h2):
    return more_general_or_equal(h1, h2) and not more_general_or_equal(h2, h1)



def min_generalization(s, x):
    """Minimal generalisation of s that covers the positive example x."""
    new = []
    for c, v in zip(s, x):
        if c == "0":
            new.append(v)
        elif c == v or c == "?":
            new.append(c)
        else:
            new.append("?")
    return tuple(new)



def min_specializations(g, x, domains):
    """Minimal specialisations of g that exclude the negative example x."""
    out = []
    for i, c in enumerate(g):
        if c == "?":
            for v in domains[i]:
                if v != x[i]:
                    out.append(g[:i] + (v,) + g[i + 1:])
    return out
```

```
# ------------------------------------------------------ main algorithm
def candidate_elimination(X, y):
    n = len(X[0])
    domains = [sorted({x[i] for x in X}) for i in range(n)]
    S = {("0",) * n}                  # most specific boundary
    G = {("?",) * n}                  # most general boundary
    trace = []

    for t, (x, positive) in enumerate(zip(X, y), start=1):
        if positive:
            G = {g for g in G if covers(g, x)}
            for s in list(S):
                if not covers(s, x):
                    S.remove(s)
                    h = min_generalization(s, x)
                    if any(more_general_or_equal(g, h) for g in G):
                        S.add(h)
            S = {s for s in S
                 if not any(strictly_more_general(s, o) for o in S)}
        else:
            S = {s for s in S if not covers(s, x)}
            for g in list(G):
                if covers(g, x):
                    G.remove(g)
                    for h in min_specializations(g, x, domains):
                        if any(more_general_or_equal(h, s) for s in S):
                            G.add(h)
            G = {g for g in G
                 if not any(strictly_more_general(o, g) for o in G)}
        trace.append((t, x, positive, set(S), set(G)))
    return S, G, trace, domains



def version_space(S, G, domains):
    """All hypotheses lying between S and G (brute-force enumeration)."""
    space = []
    for h in product(*[d + ["?"] for d in domains]):
        if any(more_general_or_equal(g, h) for g in G) and \
           any(more_general_or_equal(h, s) for s in S):
            space.append(h)
    return space



def brute_force(X, y, domains):
    """All hypotheses consistent with every example (used to verify)."""
    return [h for h in product(*[d + ["?"] for d in domains])
            if all(covers(h, x) == lab for x, lab in zip(X, y))]



# ------------------------------------------------------------ driver
if __name__ == "__main__":
    attrs, X, y = load_data(CSV_FILE)
    print("Attributes :", attrs)
    print("Examples   :", len(X), "(", sum(y), "positive,",
          len(y) - sum(y), "negative )\n")

    print("Initial  S0 =", show_set({("0",) * len(attrs)}))
    print("Initial  G0 =", show_set({("?",) * len(attrs)}), "\n")

    S, G, trace, domains = candidate_elimination(X, y)

    for t, x, positive, s_t, g_t in trace:
        label = "Positive" if positive else "Negative"
        print(f"Example {t}: {show(x)} -> {label}")
        print(f"   S{t} =", show_set(s_t))
        print(f"   G{t} =", show_set(g_t))

    print("\nFinal specific boundary S :", show_set(S))
    print("Final general  boundary G :", show_set(G))
```

```
    vs = version_space(S, G, domains)
    print(f"\nVersion space ({len(vs)} hypotheses consistent with the data):")
    for h in sorted(vs, key=lambda h: (h.count("?"), h)):
        print("   ", show(h))

    ok = sorted(vs) == sorted(brute_force(X, y, domains))
    print("\nVerification against brute-force search :", "PASSED" if ok else "FAILED")
```

10. Output

Console output of experiment_3_candidate_elimination_flu.py:

```
Attributes : ['Fever', 'Cough', 'Fatigue', 'BodyAche', 'Rash']
Examples   : 5 ( 3 positive, 2 negative )


Initial  S0 = { <0, 0, 0, 0, 0> }
Initial  G0 = { <?, ?, ?, ?, ?> }


Example 1: <High, Yes, Yes, Severe, No> -> Positive
   S1 = { <High, Yes, Yes, Severe, No> }
   G1 = { <?, ?, ?, ?, ?> }
Example 2: <High, Yes, No, Severe, No> -> Positive
   S2 = { <High, Yes, ?, Severe, No> }
   G2 = { <?, ?, ?, ?, ?> }
Example 3: <Normal, No, Yes, Mild, No> -> Negative
   S3 = { <High, Yes, ?, Severe, No> }
   G3 = { <?, ?, ?, Severe, ?>, <?, Yes, ?, ?, ?>, <High, ?, ?, ?, ?> }
Example 4: <High, Yes, Yes, Mild, Yes> -> Positive
   S4 = { <High, Yes, ?, ?, ?> }
   G4 = { <?, Yes, ?, ?, ?>, <High, ?, ?, ?, ?> }
Example 5: <Normal, Yes, No, None, No> -> Negative
   S5 = { <High, Yes, ?, ?, ?> }
   G5 = { <High, ?, ?, ?, ?> }


Final specific boundary S : { <High, Yes, ?, ?, ?> }
Final general  boundary G : { <High, ?, ?, ?, ?> }


Version space (2 hypotheses consistent with the data):
    <High, Yes, ?, ?, ?>
    <High, ?, ?, ?, ?>


Verification against brute-force search : PASSED
```

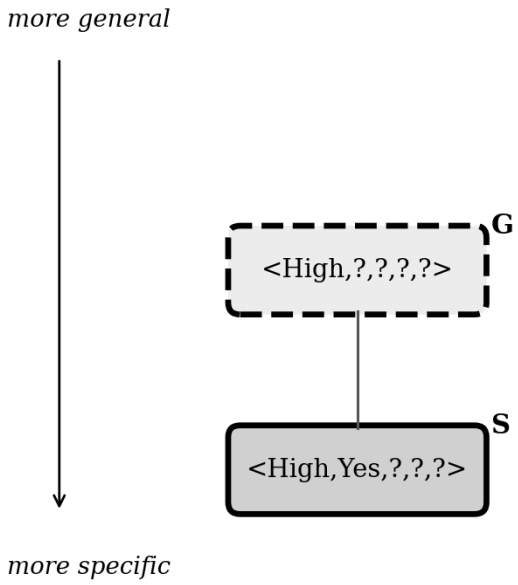


Figure 2: Version space for the Medical Diagnosis (Flu) data (S = specific boundary, G = general boundary)

11. Result / Conclusion

The Candidate-Elimination algorithm was implemented in Python and run on the Medical Diagnosis (Flu) training data (5 examples, 5 attributes). The version space contains 2 consistent hypotheses.

Final specific boundary S: { <High, Yes, ?, ?, ?> }

Final general boundary G: { <High, ?, ?, ?, ?> }

High fever is common to every positive example, so it appears in both boundaries. Because the data never shows a high-fever patient without a cough, the algorithm cannot decide whether Cough = Yes is required, and both <High, Yes, ?, ?, ?> and <High, ?, ?, ?, ?> remain consistent. One more informative example would shrink the version space further.

The version space found by the algorithm was identical to the set of hypotheses found by exhaustive search, so the implementation is verified. Hence the Candidate-Elimination algorithm outputs a description of the set of all hypotheses consistent with the training examples.

Evaluation rubric (to be filled in by the staff member)

Criteria	Max. marks	Marks awarded
Aim, concept and dataset description	10	
Algorithm, flowchart and procedure	10	
Python program (correctness and style)	15	
Output and verification	10	
Result / conclusion	5	
Total	50	

Student signature: _____ Staff signature: _____

CANDIDATE-ELIMINATION ALGORITHM

Part 4 of 5 | Data set: Loan Approval

1. Experiment No. / Date

Experiment No.: _____ (Part 4 of 5)	Date: ____ / ____ / _____

2. Experiment Question

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in Python to output a description of the set of all hypotheses consistent with the training examples.

Part 4 applies the algorithm to the Loan Approval data set (loan.csv).

3. Aim

To implement the Candidate-Elimination algorithm in Python, read the Loan Approval training examples from the CSV file loan.csv, and output the specific boundary S, the general boundary G and every hypothesis of the version space that is consistent with the training examples.

4. Concept

Concept learning means inferring a Boolean-valued function (the target concept) from labelled training examples. In this experiment a hypothesis is a conjunction of constraints on the attributes, written as a vector $<c_1, c_2, ..., c_n>$. Each constraint c_i is one of three things: a specific value (the attribute must equal that value), ? (any value is acceptable) or 0 (no value is acceptable).

A hypothesis is consistent with the training data if it classifies every training example correctly. The version space is the set of all consistent hypotheses. Instead of listing every member, the Candidate-Elimination algorithm stores only two boundaries: the specific boundary S (the most specific consistent hypotheses) and the general boundary G (the most general consistent hypotheses). Every hypothesis lying between S and G in the general-to-specific ordering also belongs to the version space.

A positive example forces S to become more general (and removes members of G that fail to cover it). A negative example forces G to become more specific (and removes members of S that wrongly cover it). In contrast to Find-S, which tracks only S, Candidate-Elimination also uses the negative examples and therefore describes all consistent hypotheses.

A simple loan-approval data set. Each row describes an applicant and whether the loan was approved. The data is illustrative and is used only to demonstrate the algorithm. Three of the four attributes have three possible values, which makes the negative examples especially informative.

5. Dataset Description

File: loan.csv. The file has one header row followed by 6 training examples (3 positive and 3 negative). The first 4 columns are the input attributes and the last column is the target: Approved (Yes = loan approved, No = rejected).

Table 1: Attributes and their values (as observed in the CSV file)

No.	Attribute	Values
1	Income	High, Low, Medium
2	CreditScore	Fair, Good, Poor
3	Employment	Salaried, SelfEmployed, Unemployed
4	Collateral	No, Yes

Table 2: Training data (loan.csv)

No.	Income	CreditScore	Employment	Collateral	Approved
1	High	Good	Salaried	Yes	Yes
2	Medium	Good	Salaried	No	Yes
3	Low	Poor	Unemployed	No	No
4	High	Fair	Salaried	Yes	No
5	Medium	Good	SelfEmployed	Yes	No
6	Low	Good	Salaried	Yes	Yes

6. Algorithm

1. Initialise S to the most specific hypothesis <0, 0, ..., 0> and G to the most general hypothesis <?, ?, ..., ?>.
2. For every training example d = <x, c(x)>, in the order given in the CSV file:

   If d is a positive example:
   - (j) Remove from G every hypothesis that does not cover x.
   - (k) For each hypothesis s in S that does not cover x: remove s from S, and add all minimal generalisations h of s such that h covers x and some member of G is more general than or equal to h.
   - (l) Remove from S every hypothesis that is more general than another hypothesis in S.

   If d is a negative example:
   - (j) Remove from S every hypothesis that covers x.
   - (k) For each hypothesis g in G that covers x: remove g from G, and add all minimal specialisations h of g such that h does not cover x and some member of S is more specific than or equal to h.
   - (l) Remove from G every hypothesis that is less general than another hypothesis in G.
3. After all examples have been processed, output S and G. The version space is every hypothesis h with $g \geq h \geq s$ for some g in G and s in S.

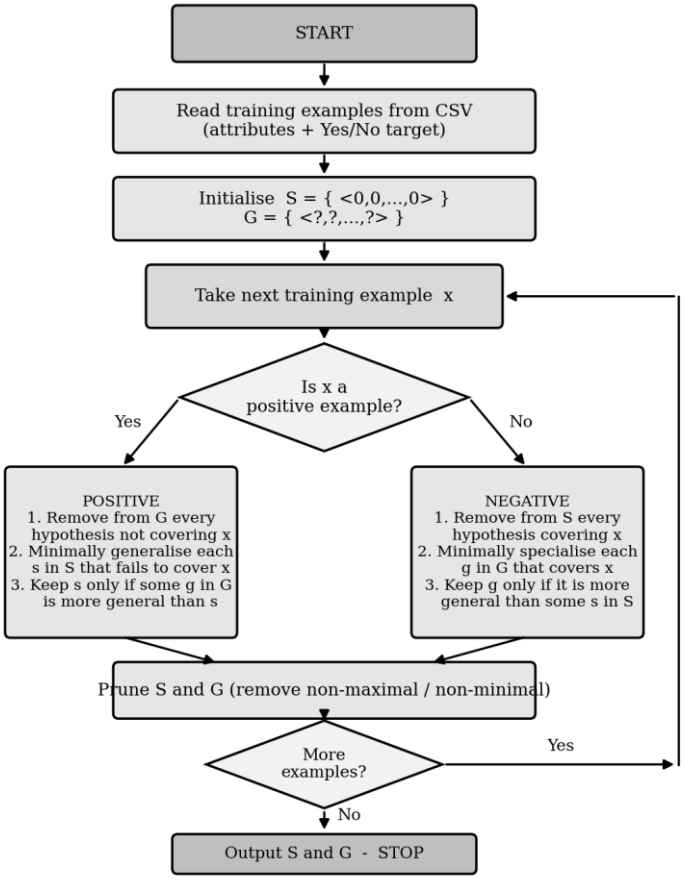


Figure 1: Flowchart of the Candidate-Elimination algorithm

7. Step-by-step Procedure

1. Create a folder for the experiment and save the training data in a CSV file named loan.csv (header row first, target column last).
2. Type the program in a Python editor (IDLE, VS Code or Jupyter) and save it as experiment_4_candidate_elimination_loan.py in the same folder as the CSV file.
3. Function load_data() reads the CSV file and separates the attribute tuples X from the Yes/No labels y.
4. Function candidate_elimination() initialises S = {<0, ..., 0>} and G = {<?, ..., ?>} and finds the domain of each attribute from the data.
5. For every example the function applies the positive-example or negative-example update given in the algorithm, using the helper functions covers(), min_generalization(), min_specializations() and more_general_or_equal().
6. The program prints S and G after every example, which gives the trace shown in Table 3.
7. Function version_space() lists every hypothesis between S and G, and brute_force() enumerates all hypotheses to confirm the same set (verification).
8. Run the program (python experiment_4_candidate_elimination_loan.py), note the output and record the final S, G and version space.

Table 3: Trace of S and G after each training example

No.	Example	Label	Specific boundary S	General boundary G
0	Initial	-	<0, 0, 0, 0>	<?, ?, ?, ?>
1	<High, Good, Salaried, Yes>	Positive	<High, Good, Salaried, Yes>	<?, ?, ?, ?>
2	<Medium, Good, Salaried, No>	Positive	<?, Good, Salaried, ?>	<?, ?, ?, ?>
3	<Low, Poor, Unemployed, No>	Negative	<?, Good, Salaried, ?>	<?, ?, Salaried, ?> <?, Good, ?, ?>
4	<High, Fair, Salaried, Yes>	Negative	<?, Good, Salaried, ?>	<?, Good, ?, ?>
5	<Medium, Good, SelfEmployed, Yes>	Negative	<?, Good, Salaried, ?>	<?, Good, Salaried, ?>
6	<Low, Good, Salaried, Yes>	Positive	<?, Good, Salaried, ?>	<?, Good, Salaried, ?>

8. Formulae

No.	Term	Expression
1	Hypothesis	$h = <c_1, c_2, ..., c_n>$, $c_i \in \{ ?, 0, \text{specific value} \}$
2	Hypothesis satisfied by x	$h(x) = 1 \Leftrightarrow$ for every i: $c_i = ?$ or $c_i = x_i$
3	Consistency	Consistent(h, D) $\Leftrightarrow$ h(x) = c(x) for every <x, c(x)> in D
4	More-general-than-or-equal	$h_j \geq_g h_k \Leftrightarrow$ for all x: ($h_k(x) = 1 \rightarrow h_j(x) = 1$)
5	Version space	VS = { h ∈ H | Consistent(h, D) }
6	Specific boundary	S = { s ∈ H | Consistent(s, D) ∧ ¬∃ s′ ∈ H [(s >g s′) ∧ Consistent(s′, D)] }
7	General boundary	G = { g ∈ H | Consistent(g, D) ∧ ¬∃ g′ ∈ H [(g′ >g g) ∧ Consistent(g′, D)] }
8	Version-space representation	VS = { h ∈ H | ∃ s ∈ S, ∃ g ∈ G : g ≥g h ≥g s }
9	Minimal generalisation (positive x)	$c_i' = x_i$ if $c_i = 0$; $c_i' = c_i$ if $c_i = x_i$ or $c_i = ?$; $c_i' = ?$ otherwise
10	Minimal specialisation (negative x)	for each i with $g_i = ?$, replace g_i by every value $v \neq x_i$ in the domain of attribute i
11	Syntactically distinct hypotheses	$\prod (\|V_i\| + 2) = (3+2)\times(3+2)\times(3+2)\times(2+2) = 500$
12	Semantically distinct hypotheses	$1 + \prod (\|V_i\| + 1) = 1 + (3+1)\times(3+1)\times(3+1)\times(2+1) = 193$

9. Python Program

File name: experiment_4_candidate_elimination_loan.py Input file: loan.csv

```
"""
Experiment 4 : Candidate-Elimination Algorithm
Dataset          : Loan Approval  (loan.csv)
```

```
Reads training examples from a CSV file (last column = target, "Yes"/"No")
and prints the version space: the specific boundary S and the general
boundary G, i.e. all hypotheses consistent with the training examples.
"""

import csv
from itertools import product

CSV_FILE = "loan.csv"



# ---------------------------------------------------------------- helpers
def load_data(path):
    """Return attribute names, list of attribute tuples, list of booleans."""
    with open(path, newline="") as f:
        rows = list(csv.reader(f))
    header, data = rows[0], rows[1:]
    X = [tuple(v.strip() for v in r[:-1]) for r in data]
    y = [r[-1].strip().lower() == "yes" for r in data]
    return header[:-1], X, y



def show(h):
    return "<" + ", ".join(h) + ">"



def show_set(hs):
    return "{ " + ", ".join(show(h) for h in sorted(hs)) + " }" if hs else "{ }"



def covers(h, x):
    """h(x) = 1 when every constraint of h is '?' or equals the value in x."""
    return all(c == "?" or c == v for c, v in zip(h, x))



def more_general_or_equal(h1, h2):
    """True if h1 >=g h2 (h1 covers everything that h2 covers)."""
    if "0" in h2:                       # h2 covers nothing
        return True
    return all(a == "?" or a == b for a, b in zip(h1, h2))



def strictly_more_general(h1, h2):
    return more_general_or_equal(h1, h2) and not more_general_or_equal(h2, h1)



def min_generalization(s, x):
    """Minimal generalisation of s that covers the positive example x."""
    new = []
    for c, v in zip(s, x):
        if c == "0":
            new.append(v)
        elif c == v or c == "?":
            new.append(c)
        else:
            new.append("?")
    return tuple(new)



def min_specializations(g, x, domains):
    """Minimal specialisations of g that exclude the negative example x."""
    out = []
    for i, c in enumerate(g):
        if c == "?":
            for v in domains[i]:
                if v != x[i]:
                    out.append(g[:i] + (v,) + g[i + 1:])
```

```
    return out



# ------------------------------------------------------ main algorithm
def candidate_elimination(X, y):
    n = len(X[0])
    domains = [sorted({x[i] for x in X}) for i in range(n)]
    S = {("0",) * n}                  # most specific boundary
    G = {("?",) * n}                  # most general boundary
    trace = []

    for t, (x, positive) in enumerate(zip(X, y), start=1):
        if positive:
            G = {g for g in G if covers(g, x)}
            for s in list(S):
                if not covers(s, x):
                    S.remove(s)
                    h = min_generalization(s, x)
                    if any(more_general_or_equal(g, h) for g in G):
                        S.add(h)
            S = {s for s in S
                 if not any(strictly_more_general(s, o) for o in S)}
        else:
            S = {s for s in S if not covers(s, x)}
            for g in list(G):
                if covers(g, x):
                    G.remove(g)
                    for h in min_specializations(g, x, domains):
                        if any(more_general_or_equal(h, s) for s in S):
                            G.add(h)
            G = {g for g in G
                 if not any(strictly_more_general(o, g) for o in G)}
        trace.append((t, x, positive, set(S), set(G)))
    return S, G, trace, domains



def version_space(S, G, domains):
    """All hypotheses lying between S and G (brute-force enumeration)."""
    space = []
    for h in product(*[d + ["?"] for d in domains]):
        if any(more_general_or_equal(g, h) for g in G) and \
           any(more_general_or_equal(h, s) for s in S):
            space.append(h)
    return space



def brute_force(X, y, domains):
    """All hypotheses consistent with every example (used to verify)."""
    return [h for h in product(*[d + ["?"] for d in domains])
            if all(covers(h, x) == lab for x, lab in zip(X, y))]



# ------------------------------------------------------------ driver
if __name__ == "__main__":
    attrs, X, y = load data(CSV FILE)
    print("Attributes :", attrs)
    print("Examples   :", len(X), "(", sum(y), "positive,",
          len(y) - sum(y), "negative )\n")

    print("Initial  S0 =", show_set({("0",) * len(attrs)}))
    print("Initial  G0 =", show_set({("?",) * len(attrs)}), "\n")

    S, G, trace, domains = candidate_elimination(X, y)

    for t, x, positive, s_t, g_t in trace:
        label = "Positive" if positive else "Negative"
        print(f"Example {t}: {show(x)} -> {label}")
        print(f"   S{t} =", show_set(s_t))
        print(f"   G{t} =", show_set(g_t))

    print("\nFinal specific boundary S :", show_set(S))
```

```
    print("Final general  boundary G :", show_set(G))

    vs = version_space(S, G, domains)
    print(f"\nVersion space ({len(vs)} hypotheses consistent with the data):")
    for h in sorted(vs, key=lambda h: (h.count("?"), h)):
        print("   ", show(h))

    ok = sorted(vs) == sorted(brute_force(X, y, domains))
    print("\nVerification against brute-force search :", "PASSED" if ok else "FAILED")
```

10. Output

Console output of experiment_4_candidate_elimination_loan.py:

```
Attributes : ['Income', 'CreditScore', 'Employment', 'Collateral']
Examples   : 6 ( 3 positive, 3 negative )

Initial  S0 = { <0, 0, 0, 0> }
Initial  G0 = { <?, ?, ?, ?> }

Example 1: <High, Good, Salaried, Yes> -> Positive
   S1 = { <High, Good, Salaried, Yes> }
   G1 = { <?, ?, ?, ?> }
Example 2: <Medium, Good, Salaried, No> -> Positive
   S2 = { <?, Good, Salaried, ?> }
   G2 = { <?, ?, ?, ?> }
Example 3: <Low, Poor, Unemployed, No> -> Negative
   S3 = { <?, Good, Salaried, ?> }
   G3 = { <?, ?, Salaried, ?>, <?, Good, ?, ?> }
Example 4: <High, Fair, Salaried, Yes> -> Negative
   S4 = { <?, Good, Salaried, ?> }
   G4 = { <?, Good, ?, ?> }
Example 5: <Medium, Good, SelfEmployed, Yes> -> Negative
   S5 = { <?, Good, Salaried, ?> }
   G5 = { <?, Good, Salaried, ?> }
Example 6: <Low, Good, Salaried, Yes> -> Positive
   S6 = { <?, Good, Salaried, ?> }
   G6 = { <?, Good, Salaried, ?> }

Final specific boundary S : { <?, Good, Salaried, ?> }
Final general  boundary G : { <?, Good, Salaried, ?> }

Version space (1 hypotheses consistent with the data):
    <?, Good, Salaried, ?>

Verification against brute-force search : PASSED
```



Figure 2: Version space for the Loan Approval data (S = specific boundary, G = general boundary)

11. Result / Conclusion

The Candidate-Elimination algorithm was implemented in Python and run on the Loan Approval training data (6 examples, 4 attributes). The version space contains 1 consistent hypothesis.

Final specific boundary S: { <?, Good, Salaried, ?> }

Final general boundary G: { <?, Good, Salaried, ?> }

The specific and general boundaries have become identical, so the version space has collapsed to a single hypothesis: a loan is approved exactly when the credit score is Good and the applicant is Salaried. When S = G the training data has identified the target concept uniquely within this hypothesis space, and no further example can change it unless the data is inconsistent.

The version space found by the algorithm was identical to the set of hypotheses found by exhaustive search, so the implementation is verified. Hence the Candidate-Elimination algorithm outputs a description of the set of all hypotheses consistent with the training examples.

Evaluation rubric (to be filled in by the staff member)

Criteria	Max. marks	Marks awarded
Aim, concept and dataset description	10	
Algorithm, flowchart and procedure	10	
Python program (correctness and style)	15	
Output and verification	10	
Result / conclusion	5	
Total	50	

Student signature: _____ Staff signature: _____

CANDIDATE-ELIMINATION ALGORITHM

Part 5 of 5 | Data set: Student Placement

1. Experiment No. / Date

Experiment No.: _____ (Part 5 of 5)	Date: ____ / ____ / _____

2. Experiment Question

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in Python to output a description of the set of all hypotheses consistent with the training examples.

Part 5 applies the algorithm to the Student Placement data set (placement.csv).

3. Aim

To implement the Candidate-Elimination algorithm in Python, read the Student Placement training examples from the CSV file placement.csv, and output the specific boundary S, the general boundary G and every hypothesis of the version space that is consistent with the training examples.

4. Concept

Concept learning means inferring a Boolean-valued function (the target concept) from labelled training examples. In this experiment a hypothesis is a conjunction of constraints on the attributes, written as a vector $<c_1, c_2, ..., c_n>$. Each constraint c_i is one of three things: a specific value (the attribute must equal that value), ? (any value is acceptable) or 0 (no value is acceptable).

A hypothesis is consistent with the training data if it classifies every training example correctly. The version space is the set of all consistent hypotheses. Instead of listing every member, the Candidate-Elimination algorithm stores only two boundaries: the specific boundary S (the most specific consistent hypotheses) and the general boundary G (the most general consistent hypotheses). Every hypothesis lying between S and G in the general-to-specific ordering also belongs to the version space.

A positive example forces S to become more general (and removes members of G that fail to cover it). A negative example forces G to become more specific (and removes members of S that wrongly cover it). In contrast to Find-S, which tracks only S, Candidate-Elimination also uses the negative examples and therefore describes all consistent hypotheses.

A campus-placement data set. Each row describes a student and whether the student was placed. The data is illustrative and is used only to demonstrate the algorithm. This is the only part in which the training data starts with a negative example.

5. Dataset Description

File: placement.csv. The file has one header row followed by 5 training examples (3 positive and 2 negative). The first 5 columns are the input attributes and the last column is the target: Placed (Yes = placed, No = not placed).

Table 1: Attributes and their values (as observed in the CSV file)

No.	Attribute	Values
1	CGPA	High, Low, Medium
2	Aptitude	Average, Good
3	Communication	Good, Poor
4	Internship	No, Yes
5	Projects	Few, Many

Table 2: Training data (placement.csv)

No.	CGPA	Aptitude	Communication	Internship	Projects	Placed
1	Low	Average	Poor	No	Few	No
2	High	Good	Good	Yes	Many	Yes
3	High	Good	Good	No	Many	Yes
4	Medium	Average	Good	Yes	Few	No
5	High	Good	Poor	Yes	Many	Yes

6. Algorithm

1. Initialise S to the most specific hypothesis <0, 0, ..., 0> and G to the most general hypothesis <?, ?, ..., ?>.
2. For every training example d = <x, c(x)>, in the order given in the CSV file:

   If d is a positive example:

   (m) Remove from G every hypothesis that does not cover x.

   (n) For each hypothesis s in S that does not cover x: remove s from S, and add all minimal generalisations h of s such that h covers x and some member of G is more general than or equal to h.

   (o) Remove from S every hypothesis that is more general than another hypothesis in S.

   If d is a negative example:

   (m) Remove from S every hypothesis that covers x.

   (n) For each hypothesis g in G that covers x: remove g from G, and add all minimal specialisations h of g such that h does not cover x and some member of S is more specific than or equal to h.

   (o) Remove from G every hypothesis that is less general than another hypothesis in G.
3. After all examples have been processed, output S and G. The version space is every hypothesis h with $g \geq h \geq s$ for some g in G and s in S.

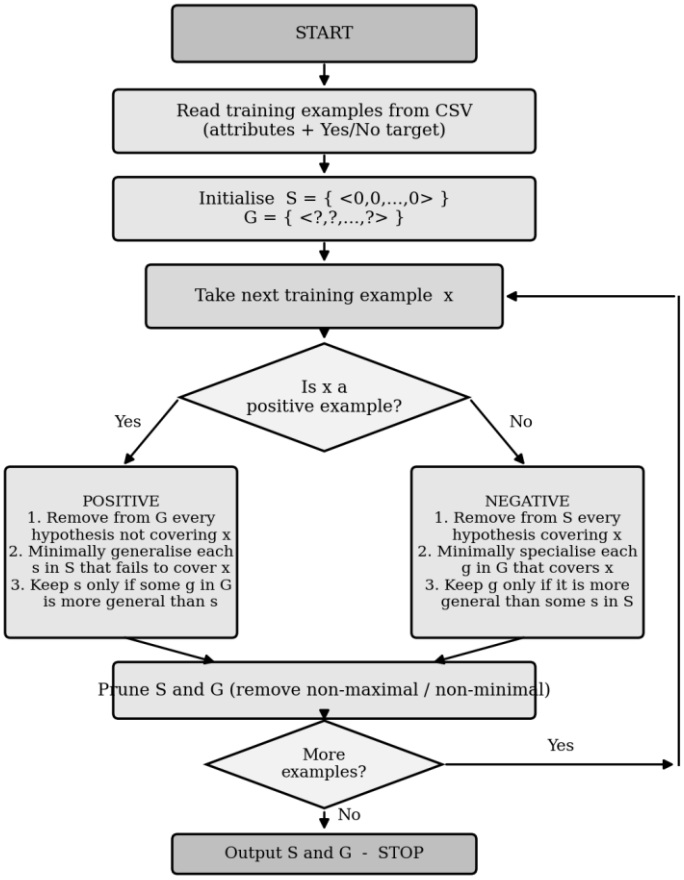


Figure 1: Flowchart of the Candidate-Elimination algorithm

7. Step-by-step Procedure

1. Create a folder for the experiment and save the training data in a CSV file named placement.csv (header row first, target column last).
2. Type the program in a Python editor (IDLE, VS Code or Jupyter) and save it as experiment_5_candidate_elimination_placement.py in the same folder as the CSV file.
3. Function load_data() reads the CSV file and separates the attribute tuples X from the Yes/No labels y.
4. Function candidate_elimination() initialises S = {<0, ..., 0>} and G = {<?, ..., ?>} and finds the domain of each attribute from the data.
5. For every example the function applies the positive-example or negative-example update given in the algorithm, using the helper functions covers(), min_generalization(), min_specializations() and more_general_or_equal().
6. The program prints S and G after every example, which gives the trace shown in Table 3.
7. Function version_space() lists every hypothesis between S and G, and brute_force() enumerates all hypotheses to confirm the same set (verification).
8. Run the program (python experiment_5_candidate_elimination_placement.py), note the output and record the final S, G and version space.

Table 3: Trace of S and G after each training example

No.	Example	Label	Specific boundary S	General boundary G
0	Initial	-	<0, 0, 0, 0, 0>	<?, ?, ?, ?, ?>
1	<Low, Average, Poor, No, Few>	Negative	<0, 0, 0, 0, 0>	<?, ?, ?, ?, Many> <?, ?, ?, Yes, ?> <?, ?, Good, ?, ?> <?, Good, ?, ?, ?> <High, ?, ?, ?, ?> <Medium, ?, ?, ?, ?>
2	<High, Good, Good, Yes, Many>	Positive	<High, Good, Good, Yes, Many>	<?, ?, ?, ?, Many> <?, ?, ?, Yes, ?> <?, ?, Good, ?, ?> <?, Good, ?, ?, ?> <High, ?, ?, ?, ?>
3	<High, Good, Good, No, Many>	Positive	<High, Good, Good, ?, Many>	<?, ?, ?, ?, Many> <?, ?, Good, ?, ?> <?, Good, ?, ?, ?> <High, ?, ?, ?, ?>
4	<Medium, Average, Good, Yes, Few>	Negative	<High, Good, Good, ?, Many>	<?, ?, ?, ?, Many> <?, Good, ?, ?, ?> <High, ?, ?, ?, ?>
5	<High, Good, Poor, Yes, Many>	Positive	<High, Good, ?, ?, Many>	<?, ?, ?, ?, Many> <?, Good, ?, ?, ?> <High, ?, ?, ?, ?>

8. Formulae

No.	Term	Expression
1	Hypothesis	$h = \langle c_1, c_2, ..., c_n \rangle$, $c_i \in \{ ?, 0, \text{specific value} \}$
2	Hypothesis satisfied by x	$h(x) = 1 \iff$ for every i: $c_i = ?$ or $c_i = x_i$
3	Consistency	Consistent(h, D) $\iff h(x) = c(x)$ for every <x, c(x)> in D
4	More-general-than-or-equal	$h_j \geq_g h_k \iff$ for all x: ($h_k(x) = 1 \rightarrow h_j(x) = 1$)
5	Version space	$VS = \{ h \in H \mid \text{Consistent}(h, D) \}$
6	Specific boundary	$S = \{ s \in H \mid \text{Consistent}(s, D) \wedge \neg\exists\, s' \in H [(s >_g s') \wedge \text{Consistent}(s', D)] \}$
7	General boundary	$G = \{ g \in H \mid \text{Consistent}(g, D) \wedge \neg\exists\, g' \in H [(g' >_g g) \wedge \text{Consistent}(g', D)] \}$
8	Version-space representation	$VS = \{ h \in H \mid \exists\, s \in S, \exists\, g \in G : g \geq_g h \geq_g s \}$
9	Minimal generalisation (positive x)	$c_i' = x_i$ if $c_i = 0$; $c_i' = c_i$ if $c_i = x_i$ or $c_i = ?$; $c_i' = ?$ otherwise
10	Minimal specialisation (negative x)	for each i with $g_i = ?$, replace g_i by every value $v \neq x_i$ in the domain of attribute i
11	Syntactically distinct hypotheses	$\prod (\lvert V_i \rvert + 2) = (3+2)\times(2+2)\times(2+2)\times(2+2)\times(2+2) = 1280$

No.	Term	Expression
12	Semantically distinct hypotheses	$1 + \prod (\|V_i\| + 1) = 1 + (3+1)\times(2+1)\times(2+1)\times(2+1)\times(2+1) = 325$

9. Python Program

File name: experiment_5_candidate_elimination_placement.py Input file: placement.csv

```
"""
Experiment 5 : Candidate-Elimination Algorithm
Dataset           : Student Placement  (placement.csv)

Reads training examples from a CSV file (last column = target, "Yes"/"No")
and prints the version space: the specific boundary S and the general
boundary G, i.e. all hypotheses consistent with the training examples.
"""


import csv
from itertools import product

CSV_FILE = "placement.csv"



# ---------------------------------------------------------------- helpers
def load_data(path):
    """Return attribute names, list of attribute tuples, list of booleans."""
    with open(path, newline="") as f:
        rows = list(csv.reader(f))
    header, data = rows[0], rows[1:]
    X = [tuple(v.strip() for v in r[:-1]) for r in data]
    y = [r[-1].strip().lower() == "yes" for r in data]
    return header[:-1], X, y



def show(h):
    return "<" + ", ".join(h) + ">"



def show_set(hs):
    return "{ " + ", ".join(show(h) for h in sorted(hs)) + " }" if hs else "{ }"



def covers(h, x):
    """h(x) = 1 when every constraint of h is '?' or equals the value in x."""
    return all(c == "?" or c == v for c, v in zip(h, x))



def more_general_or_equal(h1, h2):
    """True if h1 >=g h2 (h1 covers everything that h2 covers)."""
    if "0" in h2:                       # h2 covers nothing
        return True
    return all(a == "?" or a == b for a, b in zip(h1, h2))



def strictly_more_general(h1, h2):
    return more_general_or_equal(h1, h2) and not more_general_or_equal(h2, h1)



def min_generalization(s, x):
    """Minimal generalisation of s that covers the positive example x."""
    new = []
    for c, v in zip(s, x):
        if c == "0":
            new.append(v)
        elif c == v or c == "?":
```

```
            new.append(c)
        else:
            new.append("?")
    return tuple(new)



def min_specializations(g, x, domains):
    """Minimal specialisations of g that exclude the negative example x."""
    out = []
    for i, c in enumerate(g):
        if c == "?":
            for v in domains[i]:
                if v != x[i]:
                    out.append(g[:i] + (v,) + g[i + 1:])
    return out



# ---------------------------------------------------- main algorithm
def candidate_elimination(X, y):
    n = len(X[0])
    domains = [sorted({x[i] for x in X}) for i in range(n)]
    S = {("0",) * n}                    # most specific boundary
    G = {("?",) * n}                    # most general boundary
    trace = []

    for t, (x, positive) in enumerate(zip(X, y), start=1):
        if positive:
            G = {g for g in G if covers(g, x)}
            for s in list(S):
                if not covers(s, x):
                    S.remove(s)
                    h = min_generalization(s, x)
                    if any(more_general_or_equal(g, h) for g in G):
                        S.add(h)
            S = {s for s in S
                 if not any(strictly_more_general(s, o) for o in S)}
        else:
            S = {s for s in S if not covers(s, x)}
            for g in list(G):
                if covers(g, x):
                    G.remove(g)
                    for h in min_specializations(g, x, domains):
                        if any(more_general_or_equal(h, s) for s in S):
                            G.add(h)
            G = {g for g in G
                 if not any(strictly_more_general(o, g) for o in G)}
        trace.append((t, x, positive, set(S), set(G)))
    return S, G, trace, domains



def version space(S, G, domains):
    """All hypotheses lying between S and G (brute-force enumeration)."""
    space = []
    for h in product(*[d + ["?"] for d in domains]):
        if any(more_general_or_equal(g, h) for g in G) and \
           any(more_general_or_equal(h, s) for s in S):
            space.append(h)
    return space



def brute force(X, y, domains):
    """All hypotheses consistent with every example (used to verify)."""
    return [h for h in product(*[d + ["?"] for d in domains])
            if all(covers(h, x) == lab for x, lab in zip(X, y))]



# ------------------------------------------------------------ driver
if __name__ == "__main__":
    attrs, X, y = load_data(CSV_FILE)
    print("Attributes :", attrs)
    print("Examples   :", len(X), "(", sum(y), "positive,",
          len(y) - sum(y), "negative )\n")
```

```
    print("Initial  S0 =", show_set({("0",) * len(attrs)}))
    print("Initial  G0 =", show_set({("?",) * len(attrs)}), "\n")

    S, G, trace, domains = candidate_elimination(X, y)

    for t, x, positive, s_t, g_t in trace:
        label = "Positive" if positive else "Negative"
        print(f"Example {t}: {show(x)} -> {label}")
        print(f"   S{t} =", show_set(s_t))
        print(f"   G{t} =", show_set(g_t))

    print("\nFinal specific boundary S :", show_set(S))
    print("Final general  boundary G :", show_set(G))

    vs = version_space(S, G, domains)
    print(f"\nVersion space ({len(vs)} hypotheses consistent with the data):")
    for h in sorted(vs, key=lambda h: (h.count("?"), h)):
        print("   ", show(h))

    ok = sorted(vs) == sorted(brute_force(X, y, domains))
    print("\nVerification against brute-force search :", "PASSED" if ok else "FAILED")
```

10. Output

Console output of experiment_5_candidate_elimination_placement.py:

```
Attributes : ['CGPA', 'Aptitude', 'Communication', 'Internship', 'Projects']
Examples   : 5 ( 3 positive, 2 negative )

Initial  S0 = { <0, 0, 0, 0, 0> }
Initial  G0 = { <?, ?, ?, ?, ?> }

Example 1: <Low, Average, Poor, No, Few> -> Negative
   S1 = { <0, 0, 0, 0, 0> }
   G1 = { <?, ?, ?, ?, Many>, <?, ?, ?, Yes, ?>, <?, ?, Good, ?, ?>, <?, Good, ?, ?, ?>, <High, ?,
?, ?, ?>, <Medium, ?, ?, ?, ?> }
Example 2: <High, Good, Good, Yes, Many> -> Positive
   S2 = { <High, Good, Good, Yes, Many> }
   G2 = { <?, ?, ?, ?, Many>, <?, ?, ?, Yes, ?>, <?, ?, Good, ?, ?>, <?, Good, ?, ?, ?>, <High, ?,
?, ?, ?> }
Example 3: <High, Good, Good, No, Many> -> Positive
   S3 = { <High, Good, Good, ?, Many> }
   G3 = { <?, ?, ?, ?, Many>, <?, ?, Good, ?, ?>, <?, Good, ?, ?, ?>, <High, ?, ?, ?, ?> }
Example 4: <Medium, Average, Good, Yes, Few> -> Negative
   S4 = { <High, Good, Good, ?, Many> }
   G4 = { <?, ?, ?, ?, Many>, <?, Good, ?, ?, ?>, <High, ?, ?, ?, ?> }
Example 5: <High, Good, Poor, Yes, Many> -> Positive
   S5 = { <High, Good, ?, ?, Many> }
   G5 = { <?, ?, ?, ?, Many>, <?, Good, ?, ?, ?>, <High, ?, ?, ?, ?> }

Final specific boundary S : { <High, Good, ?, ?, Many> }
Final general  boundary G : { <?, ?, ?, ?, Many>, <?, Good, ?, ?, ?>, <High, ?, ?, ?, ?> }

Version space (7 hypotheses consistent with the data):
    <High, Good, ?, ?, Many>
    <?, Good, ?, ?, Many>
    <High, ?, ?, ?, Many>
    <High, Good, ?, ?, ?>
    <?, ?, ?, ?, Many>
    <?, Good, ?, ?, ?>
    <High, ?, ?, ?, ?>

Verification against brute-force search : PASSED
```

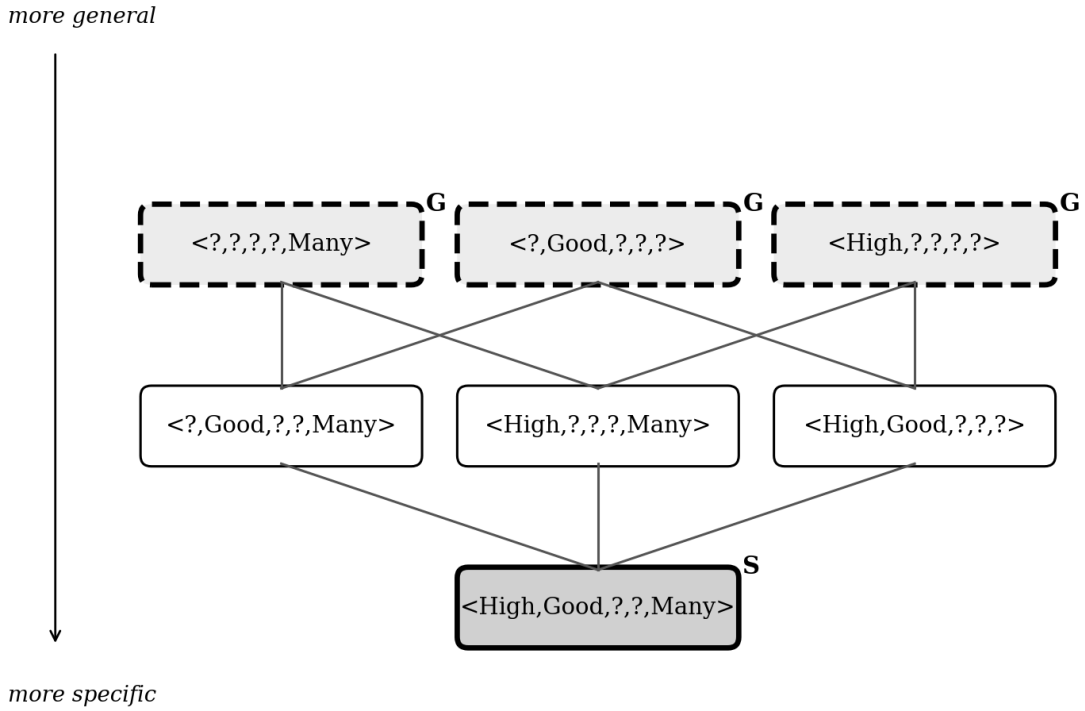


Figure 2: Version space for the Student Placement data (S = specific boundary, G = general boundary)

11. Result / Conclusion

The Candidate-Elimination algorithm was implemented in Python and run on the Student Placement training data (5 examples, 5 attributes). The version space contains 7 consistent hypotheses.

Final specific boundary S: { <High, Good, ?, ?, Many> }

Final general boundary G: { <?, ?, ?, ?, Many>, <?, Good, ?, ?, ?>, <High, ?, ?, ?, ?> }

Because the first example is negative, S stays at <0, 0, 0, 0, 0> and G is specialised into six hypotheses straight away. The positive examples that follow remove G members that do not cover them and generalise S, leaving a version space of seven hypotheses. This part demonstrates that the algorithm works for any ordering of examples.

The version space found by the algorithm was identical to the set of hypotheses found by exhaustive search, so the implementation is verified. Hence the Candidate-Elimination algorithm outputs a description of the set of all hypotheses consistent with the training examples.

Evaluation rubric (to be filled in by the staff member)

Criteria	Max. marks	Marks awarded
Aim, concept and dataset description	10	
Algorithm, flowchart and procedure	10	
Python program (correctness and style)	15	
Output and verification	10	
Result / conclusion	5	
Total	50	

Student signature: _____ Staff signature: _____